

Forward Assignment vs. Backward Assignment

- With backward assignment rule, we can get valid triple $\{q[e / v]\} v := e \{q\}$; and with forward assignment rule we get valid triple $\{p\} v := e \{p[v_0 / v] \wedge v = e[v_0 / v]\}$. The preconditions and postconditions calculated this way happen to be the weakest and strongest, respectively. Here we will show that these two rules can derive from each other. This proof will only show that these two rules are equally strong: if one is correct the other is also correct, and it does not show anything interesting between *sp* and *wlp* in general.
- First, let us calculate the $sp(q[e / v], v := e)$, where this precondition is calculated from the backward assignment rule.
 - $sp(q[e / v], v := e) \equiv q[e / v][v_0 / v] \wedge v = e[v_0 / v]$ # After $q[e / v]$, every v must be from e

$$\begin{aligned} &\Leftrightarrow q[e[v_0 / v] / v] \wedge v = e[v_0 / v] \\ &\Rightarrow q[v / v] \\ &\Leftrightarrow q \end{aligned}$$
 - $sp(q[e / v], v := e) \Rightarrow q$. Given an assignment statement S and postcondition q , then backward assignment rule + forward assignment rule can give you a postcondition that's stronger than q .
- Then, let us calculate the $wlp(v := e, p[v_0 / v] \wedge v = e[v_0 / v])$, where this postcondition is calculated from the forward assignment rule.
 - $wlp(v := e, p[v_0 / v] \wedge v = e[v_0 / v]) \equiv (p[v_0 / v] \wedge v = e[v_0 / v])[e / v]$

$$\begin{aligned} &\equiv p[v_0 / v] \wedge e = e[v_0 / v] \end{aligned}$$
 - $p \wedge v = v_0 \Leftrightarrow p \wedge T \wedge v = v_0$

$$\begin{aligned} &\Leftrightarrow p \wedge e = e \wedge v = v_0 \\ &\Leftrightarrow p[v / v] \wedge e = e[v / v] \wedge v = v_0 \\ &\Rightarrow p[v_0 / v] \wedge e = e[v_0 / v] \\ &\equiv wlp(v := e, p[v_0 / v] \wedge v = e[v_0 / v]) \end{aligned}$$
 - $(p \wedge v = v_0) \Rightarrow wlp(v := e, p[v_0 / v] \wedge v = e[v_0 / v])$. Given a precondition p (and $v = v_0$) and an assignment statement S , then forward assignment rule + backward assignment rule can give you a precondition that's weaker than $p \wedge v = v_0$.

Provability of Triples

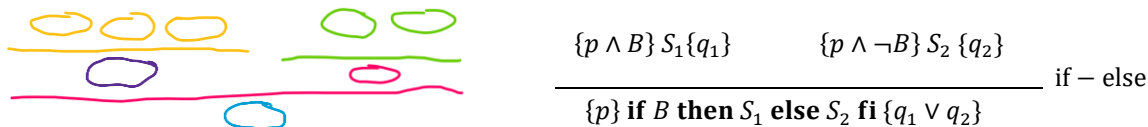
- Remember that we want to use valid correctness triples to show a program works as expected. In other words, given a program S , given a precondition p that can be provided to this program before it executes, and given a postcondition q that we expect to get after this program executes, we need to show this correctness triple $\{p\} S \{q\}$ is valid; denoted as $\models \{p\} S \{q\}$ or $\models_{tot} \{p\} S \{q\}$ (depend on what correctness level we need).
- In fact, not all triples can be decided to be valid or invalid. This is like not everything true can be proved to be true, or not all the yes-or-no questions have an algorithm that can guarantee a solution (This is discussed in computation theory).
- If a triple $\{p\} S \{q\}$ can be proved to be valid, then we say this triple is **provable**, denoted as $\vdash \{p\} S \{q\}$ or $\vdash_{tot} \{p\} S \{q\}$ (depending on what correctness level we need).

- We will focus on creating proofs for provable triples; we don't focus on deciding whether a valid triple is provable or not (it is actually undecidable).
- How to prove a triple is valid?
 - So far, we have learned many rules that can show a triple is valid, such as backward assignment rule, sequence rule, strengthen the precondition... Can you name some others?
- To prove a triple (usually with a large statement inside) being valid, we will create a **proof system of triples**, which is a set of logical formulas determined by a set of **axioms** and **rules of inference** using a set of syntactic algorithms.
 - Each true statement in a proof system is called a **judgement**, in a proof system of triples, each judgement is a **provable triple** or a **valid predicate**.
 - In math and logic, an **axiom** is something accepted to be true that is unprovable. In a proof system, some axioms can tell us some judgements are true. For example: $\models_{tot} \{p\} \text{ skip } \{p\}$ is an axiom, $x + 0 = x$ is also an axiom.
 - **Rules of Inference** are a set of rules that can combine several truths into a more complicated truth. In a proof system, rules of Inference can combine several judgements into one larger judgement. For example, modus ponens is an inference rule: $p \wedge (p \rightarrow q) \Rightarrow q$. Another example, Conditional Rule 1 is an inference rule:

$$\{p \wedge B\} S_1 \{q_1\} \wedge \{p \wedge \neg B\} S_2 \{q_2\} \Rightarrow \{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q_1 \vee q_2\}.$$

Proof Formats

- Here is an example of a proof system with Conditional Rule 1:



- The above format is called a **proof tree**. The two child judgements (**antecedents**) are above a straight line, and they together logically imply the parent judgement (**consequent**). The **rule name** is attached to the straight line.
- The advantage of proof trees is that you can read them easily. The disadvantage is that it is hard to draw since it can be wide.
- In this course, we use **Hilbert-style proofs**. Here is Conditional Rule 1 in Hilbert-style:
 1. $\{p \wedge B\} S_1 \{q_1\}$
 2. $\{p \wedge \neg B\} S_2 \{q_2\}$
 3. $\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q_1 \vee q_2\}$ if - else 1,2
 - A Hilbert-style proof has two columns. On the left, we write judgements: antecedents must appear above the consequent (not necessarily immediately above). On the right, we write the axiom we use or the rule names together with line numbers of its antecedents involved.

Some Axioms and Rules of Inference

We have seen most of these axioms and rules already.

You may assume that so far we are creating proofs under **partial correctness**, but most of the axioms and inference rules we introduce here also work for total correctness.

- **Skip Axiom:**
 1. $\{p\} \text{ skip } \{p\}$ skip
- **Backward Assignment Axiom:**
 1. $\{p[e / v]\} v := e \{p\}$ backward assignment
- **Forward Assignment Axiom:**
 1. $\{p\} v := e \{p[v_0 / v] \wedge v = e[v_0 / v]\}$ forward assignment
- **Strengthen Precondition Rule:**
 1. $p \Rightarrow p_1$ (or $p \rightarrow p_1$)
 2. $\{p_1\} S \{q\}$
 3. $\{p\} S \{q\}$ strengthen precondition 1,2
 - Each judgment in a proof is true/valid, so we can use $p \rightarrow p_1$ instead of $p \Rightarrow p_1$.
- **Weaken Postcondition Rule:**
 1. $\{p\} S \{q_1\}$
 2. $q_1 \Rightarrow q$ (or $q_1 \rightarrow q$)
 3. $\{p\} S \{q\}$ weaken postcondition 1,2
- **Sequence Rule:**
 1. $\{p\} S_1 \{r\}$
 2. $\{r\} S_2 \{q\}$
 3. $\{p\} S_1; S_2 \{q\}$ sequence 1,2
 - Remember that we previously defined a more general version of the sequence rule; we only need the postcondition in line 1 to be stronger than the precondition in line 2. Here, we define the extended sequence rule.
- **Extended Sequence Rule:**
 1. $\{p_1\} S_1 \{q_1\}$
 2. $q_1 \Rightarrow p_2$ (or $q_1 \rightarrow p_2$)
 3. $\{p_2\} S_2 \{q_2\}$
 4. $\{p_1\} S_1; S_2 \{q_2\}$ extended sequence 1,2,3
- **Conjunction Rule:**
 1. $\{p_1\} S \{q_1\}$
 2. $\{p_2\} S \{q_2\}$
 3. $\{p_1 \wedge p_2\} S \{q_1 \wedge q_2\}$ conjunction 1,2
- **Disjunction Rule:**
 1. $\{p_1\} S \{q_1\}$
 2. $\{p_2\} S \{q_2\}$
 3. $\{p_1 \vee p_2\} S \{q_1 \vee q_2\}$ disjunction 1,2

1. We know that $\{T\} x := 1; k := 0 \{x = 2^k\}$ is provable (under partial correctness). Create a formal proof for this triple.

1. $\{x = 2^0\} k := 0 \{x = 2^k\}$	backward assignment
2. $\{1 = 2^0\} x := 1 \{x = 2^0\}$	backward assignment
3. $\{1 = 2^0\} x := 1; k := 0 \{x = 2^k\}$	sequence 2,1
4. $T \rightarrow 1 = 2^0$	predicate logic
5. $\{T\} x := 1; k := 0 \{x = 2^k\}$	strengthen precondition 4,3

2. Create a different proof for the above triple by using forward assignments while proving assignment statements.

1. $\{T\} x := 1 \{x = 1\}$	forward assignment
2. $\{x = 1\} k := 0 \{x = 1 \wedge k = 0\}$	forward assignment
3. $\{T\} x := 1; k := 0 \{x = 1 \wedge k = 0\}$	sequence 1,2
4. $x = 1 \wedge k = 0 \rightarrow x = 2^k$	predicate logic
5. $\{T\} x := 1; k := 0 \{x = 2^k\}$	weaken postcondition 3,4

- **Conditional Rule 1:**

1. $\{p \wedge B\} S_1 \{q_1\}$	
2. $\{p \wedge \neg B\} S_2 \{q_2\}$	
3. $\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q_1 \vee q_2\}$	if – else 1,2

- **Conditional Rule 2:**

1. $\{p_1\} S_1 \{q_1\}$	
2. $\{p_2\} S_2 \{q_2\}$	
3. $\{(B \rightarrow p_1) \wedge (\neg B \rightarrow p_2)\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q_1 \vee q_2\}$	if – else 1,2

3. Prove Conditional Rule 2.

1. $\{p_1\} S_1 \{q_1\}$	premise / assumption
2. $(B \rightarrow p_1) \wedge B \Rightarrow p_1$	predicate logic (modus ponens)
3. $p_0 \wedge B \Rightarrow (B \rightarrow p_1) \wedge B$	predicate logic (and – elimination)
# Where $p_0 \equiv (B \rightarrow p_1) \wedge (\neg B \rightarrow p_2)$	
4. $p_0 \wedge B \Rightarrow p_1$	predicate logic
5. $\{p_0 \wedge B\} S_1 \{q_1\}$	strengthen precondition 4, 1
6. $\{p_2\} S_2 \{q_2\}$	premise
7. $(\neg B \rightarrow p_2) \wedge \neg B \Rightarrow p_2$	predicate logic
8. $p_0 \wedge \neg B \Rightarrow (\neg B \rightarrow p_2) \wedge \neg B$	predicate logic
9. $p_0 \wedge \neg B \Rightarrow p_2$	predicate logic
10. $\{p_0 \wedge \neg B\} S_2 \{q_2\}$	strengthen precondition 9, 6
11. $\{p_0\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q_1 \vee q_2\}$	if – else 5,10
# Where $p_0 \equiv (B \rightarrow p_1) \wedge (\neg B \rightarrow p_2)$	

4. Use conditional rule 1 to create proof for an **if – then** statement.

1. $\{p \wedge B\} S_1 \{q_1\}$	premise
2. $\{p \wedge \neg B\} \text{ skip } \{p \wedge \neg B\}$	skip
3. $\{p\} \text{ if } B \text{ then } S_1 \text{ fi } \{q_1 \vee p \wedge \neg B\}$	if – else 1,2

- **Nondeterministic Conditional Rule:**

1. $\{p \wedge B_1\} S_1 \{q_1\}$	
2. $\{p \wedge B_2\} S_2 \{q_2\}$	
3. $\{p\} \text{ if } B_1 \rightarrow S_1 \square B_2 \rightarrow S_2 \text{ fi } \{q_1 \vee q_2\}$	if – fi 1,2